TP12 – Carpeta técnica del proyecto y demo funcional

OficioRed · Grupo 12 · Prácticas Profesionalizantes II – Taller · Ciclo 2026

PROPÓSITO

Consolidar la documentación técnica del proyecto, registrar el estado real alcanzado y preparar una demostración funcional coherente con la versión implementada. No se inició ninguna feature nueva para completar esta carpeta.

Índice

A. Datos generales 2

B. Resumen ejecutivo del proyecto 2

B.1. MVP/PMF original, prototipo actual y brecha 2

C. Versión estable seleccionada 3

D. Funcionalidades y estado actual 3

E. Arquitectura y flujo principal 5

F. Componentes técnicos principales 6

G. Modelo de datos / persistencia 7

H. Repositorio y README 9

I. Guía de instalación y ejecución 9

J. Trazabilidad del recorrido TP3–TP11 9

K. Pruebas funcionales y evidencias 10

L. Incidencias y pendientes técnicos 13

M. Uso de IA durante el proyecto 13

N. Decisiones técnicas relevantes 15

O. Preparación de la demostración final 16

O.1. Guion cronometrado 16

P. Ensayo y verificación previa 17

Q. Evidencias a incorporar 17

R. Autoevaluación final del grupo 21

S. Checklist antes de entregar el TP12 21

T. Entrega 22

A. Datos generales

Nombre del proyecto: OficioRed

Grupo N.º: 12

Integrantes: Juan Leistmeister · Lisandro · Mirian

Usuario principal: Vecino/cliente que busca un profesional de oficios en El Colorado (Formosa)

Feature / línea funcional trabajada: Directorio de profesionales de oficios con búsqueda, perfiles públicos, gestión de profesionales y panel de administración

Rama / commit / versión de referencia: Rama main · Commit 0f3928dc4f3dc7af54e2f7d9521138d3df0ada1d

Repositorio o ubicación: Repositorio local Git en C:\xampp\htdocs\oficiored (desplegado en InfinityFree)

Fecha: 09/10/2026

B. Resumen ejecutivo del proyecto

Problema o necesidad que aborda:

En El Colorado (Formosa) no existe un canal digital local que permita a los vecinos encontrar rápidamente electricistas, plomeros, albañiles u otros trabajadores independientes de la zona. El contacto se da por recomendaciones informales o grupos de WhatsApp, sin garantía de disponibilidad ni datos verificados.

Solución propuesta:

OficioRed es un directorio web de profesionales de oficios que permite a los vecinos buscar y contactar trabajadores por categoría y zona de cobertura, y a los propios trabajadores publicar y gestionar su perfil de forma gratuita. Un panel de administración modera los perfiles antes de que sean públicos. El contacto es directo (teléfono / email / WhatsApp), sin intermediarios ni comisiones.

Usuario principal y finalidad:

Vecinos de El Colorado que necesitan contratar un servicio puntual (cliente), profesionales independientes que quieren conseguir más trabajo (profesional) y administradores del sitio que gestionan el directorio (admin).

B.1. MVP/PMF original, prototipo actual y brecha

MVP/PMF original (TP3):

- Búsqueda de profesionales por oficio
- Filtro por categoría
- Perfil público del profesional
- Registro de profesional
- Login / autenticación
- Dashboard del profesional
- Edición de perfil y foto
- Gestión de servicios ofrecidos
- Panel de administración (aprobar/rechazar)
- Gestión de categorías
- Contacto directo (teléfono/email)
- Formulario de solicitud de presupuesto

Prototipo implementado hasta TP12:

Todas las funcionalidades del MVP están implementadas y validadas incluyendo el formulario de solicitud de presupuesto (INC-01 resuelto, oct 2026). Además se incorporaron, por fuera del MVP (Fase 2): recuperación de contraseña con token, verificación de identidad (DNI), panel de apariencia del sitio, gestión de zonas de cobertura y barra de categorías sticky.

Brecha actual:

Formulario de solicitud de presupuesto: funcionalidad imprescindible del MVP original que implementado y validado (INC-01 resuelto oct 2026). El MVP original está completo. No existe brecha funcional pendiente.

C. Versión estable seleccionada

Mecanismo de respaldo: Git (repositorio local)

Rama o copia: main

Commit / tag / versión: 0f3928dc4f3dc7af54e2f7d9521138d3df0ada1d

Fecha de la versión: 09/10/2026

Motivo por el que se considera estable: Es el último commit integrado tras la validación funcional de TP11 (pruebas PF-1 a PF-4 aprobadas). Los tres flujos principales fueron verificados en entorno local sin errores críticos.

D. Funcionalidades y estado actual

Una ficha por funcionalidad o módulo relevante.

Funcionalidad 1: Búsqueda por texto (oficio/nombre)

Usuario: Cliente

Estado: Implementado y validado

Evidencia: E1, E2

Observaciones: buscar.php con PDO y filtro estado='activo'

Funcionalidad 2: Filtro por categoría

Usuario: Cliente

Estado: Implementado y validado

Evidencia: E2

Observaciones: Usa id_categoria (no texto)

Funcionalidad 3: Perfil público

Usuario: Cliente

Estado: Implementado y validado

Evidencia: E3

Observaciones: perfil.php con contacto directo (tel/email/WhatsApp)

Funcionalidad 4: Registro de profesional

Usuario: Profesional

Estado: Implementado y validado

Evidencia: —

Observaciones: Con validaciones y estado inicial 'en_revision'

Funcionalidad 5: Login / sesión

Usuario: Profesional / Admin

Estado: Implementado y validado

Evidencia: E4

Observaciones: bcrypt + CSRF + regeneración de session ID

Funcionalidad 6: Dashboard del profesional

Usuario: Profesional

Estado: Implementado y validado

Evidencia: E4

Observaciones: dashboard.php

Funcionalidad 7: Edición de perfil y foto

Usuario: Profesional

Estado: Implementado y validado

Evidencia: —

Observaciones: Validación de tipo y tamaño de imagen

Funcionalidad 8: Gestión de servicios

Usuario: Profesional

Estado: Implementado y validado

Evidencia: —

Observaciones: mis-servicios.php, CRUD completo

Funcionalidad 9: Recuperación de contraseña

Usuario: Profesional

Estado: Implementado y validado

Evidencia: —

Observaciones: Token con expiración 1h, email vía PHPMailer. Incorporación Fase 2 (no MVP)

Funcionalidad 10: Panel admin – profesionales

Usuario: Admin

Estado: Implementado y validado

Evidencia: E5

Observaciones: Aprobar / Rechazar / Activar / Eliminar

Funcionalidad 11: Panel admin – categorías

Usuario: Admin

Estado: Implementado y validado

Evidencia: —

Observaciones: CRUD con icono picker (Lucide)

Funcionalidad 12: Panel admin – zonas

Usuario: Admin

Estado: Implementado y validado

Evidencia: —

Observaciones: Activar / Desactivar

Funcionalidad 13: Panel admin – verificaciones DNI

Usuario: Admin

Estado: Implementado y validado

Evidencia: —

Observaciones: Cola de pendientes

Funcionalidad 14: Panel admin – apariencia

Usuario: Admin

Estado: Implementado y validado

Evidencia: E6

Observaciones: Configura hero/CTA/texto del sitio

Funcionalidad 15: Formulario de solicitud de presupuesto

Usuario: Cliente

Estado: Implementado y validado

Evidencia: E7

Observaciones: INC-01 resuelto oct 2026

E. Arquitectura y flujo principal

Inicio del flujo: El usuario accede a index.php o escribe en el buscador

Componentes involucrados: index.php, buscar.php, includes/db.php, includes/funciones.php, includes/card-profesional.php, PDO, MariaDB

Datos que circulan: Parámetros GET (q, cat, zona); resultados de consulta SQL; HTML renderizado en el servidor

Dependencias principales: PHP 8, MariaDB 10.4, Apache (XAMPP), PDO

Fin del flujo / resultado observable: Grid de tarjetas de profesionales con nombre, oficio, zona, contacto y botón 'Ver perfil'

Flujo principal (cliente):

```
Cliente (navegador)
  -> index.php / buscar.php  (GET: q, cat, zona)
```

```
  -> PDO -> MariaDB (profesionales JOIN categorias JOIN zonas, WHERE estado='activo')
  -> includes/card-profesional.php (parcial reutilizable)
  -> Respuesta HTML al navegador
```

Flujo secundario (registro y moderación):

```
Profesional -> registro.php (POST) -> INSERT profesionales (estado='en_revision')
  -> Admin -> admin/profesionales.php -> Aprobar -> UPDATE estado='activo' + email de bienvenida
  -> Perfil visible en búsquedas públicas
```

Flujo de solicitud de presupuesto (cliente → profesional): Cliente (navegador) → perfil.php (formulario de contacto) → POST solicitar-presupuesto.php → validación PHP + PDO INSERT solicitudes_presupuesto + PHPMailer notifica al profesional por email → dashboard.php (profesional consulta solicitudes recibidas, con indicador de no leídas y acciones de respuesta)

F. Componentes técnicos principales

Componente 1: index.php / buscar.php / perfil.php

Tipo/capa: Frontend / controlador

Responsabilidad: Páginas públicas: inicio, búsqueda y perfil

Tecnología/dependencia: PHP 8

Componente 2: registro.php / login.php / dashboard.php / editar-perfil.php

Tipo/capa: Frontend / controlador

Responsabilidad: Alta, autenticación y área privada del profesional

Tecnología/dependencia: PHP 8, sesiones

Componente 3: mis-servicios.php / recuperacion.php / nueva-password.php

Tipo/capa: Frontend / controlador

Responsabilidad: Servicios del profesional y recuperación de contraseña

Tecnología/dependencia: PHP 8, PHPMailer

Componente 4: admin/ (profesionales, categorias, zonas, verificaciones, apariencia)

Tipo/capa: Frontend / controlador

Responsabilidad: Panel de administración con login propio

Tecnología/dependencia: PHP 8

Componente 5: includes/db.php

Tipo/capa: Infraestructura

Responsabilidad: Conexión PDO a MariaDB + política de errores

Tecnología/dependencia: PDO

Componente 6: includes/funciones.php

Tipo/capa: Lógica compartida

Responsabilidad: sanitizar(), csrf_token(), validaciones, helpers

Tecnología/dependencia: PHP 8

Componente 7: includes/mailer.php

Tipo/capa: Infraestructura

Responsabilidad: Envío de emails

Tecnología/dependencia: PHPMailer / SMTP

Componente 8: includes/card-profesional.php

Tipo/capa: Vista (parcial)

Responsabilidad: Tarjeta reutilizable de profesional

Tecnología/dependencia: HTML/PHP

Componente 9: css/estilos.css · css/admin.css · js/main.js

Tipo/capa: Presentación

Responsabilidad: Estilos del sitio y del admin; comportamientos

Tecnología/dependencia: CSS3 / JS vanilla

Componente 10: sql/*.sql

Tipo/capa: Base de datos

Responsabilidad: Scripts de migración por versión

Tecnología/dependencia: MariaDB

Componente 11: solicitar-presupuesto.php / dashboard.php (sección solicitudes)

Tipo/capa: Frontend / controlador

Responsabilidad: Recibe y valida la solicitud de presupuesto del cliente, la persiste en solicitudes_presupuesto y notifica al profesional por email; el dashboard permite consultar, filtrar y eliminar solicitudes recibidas

Tecnología/dependencia: PHP 8, PDO, PHPMailer

G. Modelo de datos / persistencia

Base de datos relacional MariaDB/MySQL, charset utf8mb4, acceso con PDO (prepared statements).

Tabla / entidad 1: profesionales

Propósito: Perfil de cada profesional

Datos relevantes: id, nombre, apellido, email, telefono, oficio, foto_perfil, estado, dni_verificado, fecha_registro

Claves / relaciones / restricciones: FK categoria_id -> categorias

Tabla / entidad 2: categorias

Propósito: Categorías de oficio

Datos relevantes: id_categoria, nombre, descripcion, icono_lucide, orden

Claves / relaciones / restricciones: nombre único

Tabla / entidad 3: zonas / profesional_zonas

Propósito: Zonas de cobertura y su relación con el profesional

Datos relevantes: id_zona, nombre, activa, es_todo / profesional_id, zona_id

Claves / relaciones / restricciones: FK a profesionales y zonas

Tabla / entidad 4: servicios

Propósito: Servicios específicos ofrecidos

Datos relevantes: id, profesional_id, nombre, descripcion, precio_desde, activo

Claves / relaciones / restricciones: FK -> profesionales

Tabla / entidad 5: usuarios

Propósito: Credenciales de acceso (admin y profesionales)

Datos relevantes: id, email, contrasena (hash bcrypt), id_profesional, rol

Claves / relaciones / restricciones: rol in {admin, profesional}

Tabla / entidad 6: tokens_recuperacion

Propósito: Tokens temporales de reset de contraseña

Datos relevantes: token (hash), id_usuario, expiracion, usado

Claves / relaciones / restricciones: FK -> usuarios; expiración 1h

Tabla / entidad 7: configuracion_sitio

Propósito: Configuración visual del sitio (hero/CTA/textos)

Datos relevantes: clave, valor

Claves / relaciones / restricciones: clave única

Tabla / entidad 8: solicitudes_presupuesto

Propósito: Solicitudes de presupuesto enviadas por clientes a un profesional específico

Datos relevantes: id, profesional_id, nombre_cliente, telefono_cliente, email_cliente, modalidad (ENUM presencial/digital), zona_servicio, descripcion (TEXT), leida (TINYINT), fecha (DATETIME)

Claves / relaciones / restricciones: FK profesional_id → profesionales (ON DELETE CASCADE); índice idx_profesional; charset utf8mb4

Consulta principal del buscador (buscar.php):

```
SELECT p.id_profesional, p.nombre, p.apellido, p.oficio, p.zona_cobertura,
       p.telefono, p.email, p.foto_perfil, p.dni_verificado, c.nombre AS categoria_nombre
FROM profesionales p
LEFT JOIN categorias c ON c.id_categoria = p.categoria_id
WHERE p.estado = 'activo'
  AND (p.oficio LIKE ? OR c.nombre LIKE ? OR p.nombre LIKE ? OR p.apellido LIKE ?)
ORDER BY p.nombre ASC LIMIT 12 OFFSET ?
```

La condición AND p.estado = 'activo' es obligatoria en todas las consultas públicas.

H. Repositorio y README

URL o ubicación del repositorio: Repositorio local Git en C:\xampp\htdocs\oficiored (rama main, commit 0f3928d). Desplegado en InfinityFree

README ubicado en: Raíz del proyecto (/README.md) — creado

¿Describe qué hace el proyecto?: Sí. Describe OficioRed, su propósito y usuarios

¿Incluye instalación y ejecución?: Sí. Pasos con XAMPP, importación de SQL y configuración de credenciales

¿Incluye tecnologías y estructura?: Sí. Stack (PHP 8, MariaDB, PDO) y árbol de archivos

¿Incluye estado, limitaciones y equipo?: Sí. Estado actual, pendientes/brecha y equipo (Grupo 12)

I. Guía de instalación y ejecución

Paso 1 – Requisitos previos: XAMPP (PHP 8.x + Apache + MariaDB 10.4).

Paso 2 – Preparar proyecto/carpeta: Copiar el proyecto en C:\xampp\htdocs\oficiored\ e iniciar Apache y MySQL.

Paso 3 – Instalar dependencias: No requiere Composer. PHPMailer incluido; Lucide por CDN.

Paso 4 – Preparar base de datos: Crear la BD 'oficiored' e importar en orden: oficiored_v1.sql, v2_fase1_*.sql, v2_fase2_*.sql, v2_fase3_zonas.sql, v3_configuracion_sitio.sql, v4_solicitudes_presupuesto.sql (y opcional datos_prueba.sql).

Paso 5 – Configurar variables o archivos: Copiar includes/config.example.php como includes/config.local.php y completar host, base, usuario, password, charset y app_debug (SMTP si se usa recuperación).

Paso 6 – Iniciar servicios/servidor: Apache y MySQL en verde en el panel de XAMPP.

Paso 7 – Acceder al sistema: Sitio: http://localhost/oficiored/ · Admin: http://localhost/oficiored/admin/

Paso 8 – Datos de prueba ficticios: La demo usa perfiles de prueba (datos ficticios) cargados para la ocasión; incluye 1 profesional en 'en_revision'.

IMPORTANTE: no se incluyen contraseñas reales, tokens ni secretos. config.local.php está excluido en .gitignore; solo se documentan las variables necesarias.

J. Trazabilidad del recorrido TP3–TP11

TP3: Definición del proyecto: directorio de oficios; backlog HU1–HU7.

Qué se conserva hoy: HU de búsqueda, registro, perfil, login, dashboard y panel admin implementadas.

Referencia/evidencia: GRUPO_13_BACKLOG_HISTORIAS_USUARIO_PP2_REENTREGA.docx

TP4: Feature spec del módulo de búsqueda (inputs, filtrado, cards).

Qué se conserva hoy: buscar.php y card-profesional.php derivan de esa spec.

Referencia/evidencia: /buscar.php, /includes/card-profesional.php

TP5: Descomposición técnica: PHP monolítico por páginas, MVC-lite con includes/.

Qué se conserva hoy: Estructura actual del proyecto.

Referencia/evidencia: Árbol de archivos (sección F)

TP6: Prompt técnico: migrar de MySQLi a PDO.

Qué se conserva hoy: Toda la capa de datos usa PDO con prepared statements.

Referencia/evidencia: /includes/db.php

TP7: Implementación asistida del buscador con filtros GET y render server-side.

Qué se conserva hoy: buscar.php con q/cat/zona e include de la card.

Referencia/evidencia: /buscar.php

TP8: Autenticación con bcrypt, CSRF y regeneración de session ID.

Qué se conserva hoy: login.php con password_verify + csrf_token + session_regenerate_id.

Referencia/evidencia: /login.php, /includes/funciones.php

TP9: Seguridad transversal: sanitizar(), validación de foto, patrón PRG.

Qué se conserva hoy: Funciones en funciones.php usadas en todo el proyecto.

Referencia/evidencia: /includes/funciones.php

TP10: Integración: panel admin con lista, filtros, aprobación/rechazo y CRUD de categorías.

Qué se conserva hoy: admin/*.php funcionando.

Referencia/evidencia: /admin/*.php

TP11: Validación del prototipo: PF-1 a PF-4 aprobadas; correcciones de sesión/estado.

Qué se conserva hoy: Flujo de los 3 actores validado sin errores críticos.

Referencia/evidencia: Sección K de esta carpeta

K. Pruebas funcionales y evidencias

Caso PF-1: Válido

Resultado esperado: Resultados filtrados con estado='activo'

Resultado obtenido: Resultados correctos con el filtro activo

Estado: Cumple

Evidencia: E1

Caso PF-2: Válido

Resultado esperado: Cards de la categoría seleccionada

Resultado obtenido: Chips activos y resultados filtrados

Estado: Cumple

Evidencia: E2

Caso PF-3: Válido

Resultado esperado: Registro con estado inicial 'en_revision'

Resultado obtenido: Registro exitoso, estado correcto

Estado: Cumple

Evidencia: —

Caso PF-4: Válido

Resultado esperado: Login inicia sesión y redirige a dashboard

Resultado obtenido: Login correcto con regeneración de session ID

Estado: Cumple

Evidencia: E4

Caso PF-5: Válido

Resultado esperado: Perfil público con datos y avatar/foto

Resultado obtenido: Perfil renderizado con todos los datos

Estado: Cumple

Evidencia: E3

Caso PF-6: Válido

Resultado esperado: Aprobación: estado pasa a 'activo' + email

Resultado obtenido: Estado actualizado y email enviado

Estado: Cumple

Evidencia: E5

Caso PF-7: Válido

Resultado esperado: Email de recuperación con enlace de un solo uso (1h)

Resultado obtenido: Email recibido, enlace funcional

Estado: Cumple

Evidencia: —

Caso PF-8: Válido

Resultado esperado: Edición de perfil y reemplazo de foto

Resultado obtenido: Edición persistida, previsualización OK

Estado: Cumple

Evidencia: —

Caso PF-9: Alternativo (seguridad)

Resultado esperado: Acceso a admin sin sesión redirige a login

Resultado obtenido: Redirige correctamente

Estado: Cumple

Evidencia: —

Caso PF-10: Inválido (seguridad)

Resultado esperado: Inyección SQL en el buscador sin efecto

Resultado obtenido: PDO bloquea la inyección

Estado: Cumple

Evidencia: —

Caso PF-11: Válido

Resultado esperado: Cambios de apariencia (hero/CTA) visibles en index

Resultado obtenido: Cambios visibles inmediatamente

Estado: Cumple

Evidencia: E6

Caso PF-12: Alternativo (seguridad)

Resultado esperado: Profesional inactivo no aparece en búsquedas

Resultado obtenido: No figura

Estado: Cumple

Evidencia: —

Caso PF-13: Válido (solicitud de presupuesto con datos correctos)

Resultado esperado: Solicitud guardada en solicitudes_presupuesto y mensaje de confirmación al cliente

Resultado obtenido: Registro persistido correctamente; email enviado al profesional; mensaje de éxito mostrado al cliente

Estado: Cumple

Evidencia: E7

Caso PF-14: Inválido (datos faltantes o incorrectos en el formulario de presupuesto)

Resultado esperado: Rechazo con mensaje de error; sin inserción en la base de datos

Resultado obtenido: Validación PHP rechaza el envío; redirige a perfil.php con mensaje de error en sesión; no se genera ningún registro

Estado: Cumple

Evidencia: —

Caso PF-15: Alternativo (consulta de solicitudes recibidas por profesional autenticado)

Resultado esperado: El profesional ve en su dashboard el listado de solicitudes recibidas con indicador de no leídas y acciones de gestión

Resultado obtenido: Panel de solicitudes visible en dashboard.php con filtros por estado (todas/nuevas) y opción de eliminación

Estado: Cumple

Evidencia: —

L. Incidencias y pendientes técnicos

INC-01: Formulario de solicitud de presupuesto

Tipo: Pendiente técnico

Severidad: Media

Prioridad: Alta

Estado: Resuelto

Próxima acción: Validado e integrado. No requiere nueva implementación.

INC-02: README.md no existía en el repositorio

Tipo: Incidencia

Severidad: Baja

Prioridad: Media

Estado: Corregida

Próxima acción: README.md creado en la raíz

INC-07: Evidencia E5 no muestra el flujo de aprobación 'en_revision'

Tipo: Incidencia

Severidad: Baja

Prioridad: Media

Estado: Resuelto

Próxima acción: E5 reemplazada por captura del flujo de moderación (oct 2026)

PEND-01: Sistema de reseñas

Tipo: Pendiente técnico

Severidad: Baja

Prioridad: Baja

Estado: Pendiente (Fase 2)

Próxima acción: Requiere nueva HU documentada

PEND-02: Deploy en producción

Tipo: Pendiente técnico

Severidad: Baja

Prioridad: Baja

Estado: En curso

Próxima acción: Desplegado en InfinityFree; verificar flujo completo en producción

M. Uso de IA durante el proyecto

Uso 1: TP5 – Estructura del proyecto

Problema planteado: Cómo organizar un proyecto PHP sin framework

Contexto proporcionado a la IA: Stack y objetivo del proyecto

Aporte recibido: Estructura con includes/ y páginas por funcionalidad (MVC-lite)

Decisión del equipo: Se adoptó; se eliminaron subcarpetas innecesarias

Verificación realizada: Revisión manual del árbol

Uso 2: TP6 – Migración a PDO

Problema planteado: Migrar de MySQLi a PDO con prepared statements

Contexto proporcionado a la IA: Consultas existentes

Aporte recibido: Plantillas de conexión y prepare()/execute()

Decisión del equipo: Se adoptó PDO en todo el proyecto

Verificación realizada: Pruebas con caracteres especiales

Uso 3: TP7 – Módulo de búsqueda

Problema planteado: Filtros combinados (texto + categoría + zona)

Contexto proporcionado a la IA: Esquema de datos

Aporte recibido: Consulta SQL base con condicionales

Decisión del equipo: Se revisó y se corrigió estado='activo'

Verificación realizada: PF-1 y PF-2

Uso 4: TP8 – Autenticación

Problema planteado: Login seguro con sesiones en PHP

Contexto proporcionado a la IA: Flujo de login

Aporte recibido: bcrypt + CSRF + regeneración de session ID

Decisión del equipo: Se adoptó el patrón completo

Verificación realizada: PF-4, PF-9

Uso 5: TP9 – Seguridad transversal

Problema planteado: Validación, sanitización y PRG

Contexto proporcionado a la IA: Formularios del proyecto

Aporte recibido: sanitizar() y patrón PRG consistente

Decisión del equipo: Se aplicó en todos los formularios

Verificación realizada: PF-10 y revisión de código

Uso 6: TP10 – Panel admin

Problema planteado: CRUD con acciones de estado

Contexto proporcionado a la IA: Requerimientos de moderación

Aporte recibido: Estructura de tabla + formularios de acción

Decisión del equipo: Se ajustó el diseño y se agregaron confirmaciones JS

Verificación realizada: PF-6

Uso 7: Fase 2 – Recuperación de contraseña

Problema planteado: Tokens con expiración y envío de email

Contexto proporcionado a la IA: Modelo de usuarios

Aporte recibido: Tabla de tokens y lógica de reset

Decisión del equipo: Se adoptó con expiración a 1 hora

Verificación realizada: PF-7

Uso 8: Fase 2 – Panel de apariencia

Problema planteado: Configuración dinámica de hero/CTA

Contexto proporcionado a la IA: Necesidad de editar textos sin redeploy

Aporte recibido: Tabla clave-valor configuracion_sitio

Decisión del equipo: Se adoptó; se agregaron sliders de posición

Verificación realizada: PF-11

N. Decisiones técnicas relevantes

Decisión 1: PHP monolítico por páginas (sin framework ni routing)

Alternativa considerada: Laravel / Symfony

Justificación: Reducir la curva de aprendizaje y mantener la pila al alcance del equipo

Impacto actual: Simplicidad de deploy; cada página es autónoma y fácil de depurar

Decisión 2: PDO con prepared statements (migrado desde MySQLi)

Alternativa considerada: MySQLi procedural

Justificación: Portabilidad y protección nativa contra inyección SQL

Impacto actual: Capa de datos uniforme y segura

Decisión 3: Parcial card-profesional.php reutilizable

Alternativa considerada: Duplicar HTML en cada página

Justificación: DRY: un solo archivo controla la tarjeta

Impacto actual: Cambios visuales en un solo lugar

Decisión 4: categorias y zonas como tablas (no arrays hardcodeados)

Alternativa considerada: Constantes en PHP

Justificación: Que el admin edite categorías y zonas sin tocar código

Impacto actual: Extensibilidad real desde el panel

Decisión 5: Estado 'en_revision' intermedio para altas

Alternativa considerada: Aprobar automáticamente

Justificación: Que solo perfiles revisados aparezcan en el directorio

Impacto actual: Moderación manual preserva la calidad

Decisión 6: CSRF con hash_equals en todos los POST

Alternativa considerada: Sin protección / comparación insegura

Justificación: Evitar ataques de timing y CSRF

Impacto actual: Todos los formularios protegidos

Decisión 7: Credenciales fuera del código (config.local.php)

Alternativa considerada: Credenciales hardcodeadas

Justificación: No versionar secretos; facilitar el deploy

Impacto actual: Permite pasar de XAMPP a InfinityFree sin tocar código

O. Preparación de la demostración final

Duración objetivo: 5 a 7 minutos

Versión que se demostrará: Rama main, commit 0f3928d

Flujo principal elegido: Cliente busca un electricista -> ve el perfil -> el profesional gestiona su perfil -> el admin aprueba un nuevo perfil

Datos de prueba preparados: Al menos 4 profesionales activos con distintas categorías y zonas; 1 profesional en 'en_revision' para la demo de aprobación (datos ficticios)

Caso alternativo o validación significativa: Mostrar que un profesional inactivo no aparece en búsquedas (PF-12)

Integrante que opera: Lisandro

Integrantes que explican: Juan (cliente, arquitectura, moderación y decisión técnica) y Mirian (profesional, admin y cierre)

Plan de contingencia: Video del flujo + capturas explicadas + copia local estable + backup .sql

O.1. Guion cronometrado

0:00–0:40: Problema, usuario y propósito

Qué se dice: OficioRed resuelve cómo encontrar un profesional de oficios en El Colorado, hoy por WhatsApp o boca a boca. Es un directorio web local.

Qué se muestra: index.php con hero y categorías

Responsable: Juan

0:40–1:10: Versión y alcance actual

Qué se dice: Stack PHP 8, MariaDB, PDO, sin framework. Versión main 0f3928d. Alcance: 3 flujos (cliente, profesional, admin).

Qué se muestra: Pantalla de inicio / referencia a la versión

Responsable: Juan

1:10–4:30: Flujo funcional principal

Qué se dice: Un vecino busca por texto o categoría, entra al perfil y ve el contacto directo. Luego, el profesional gestiona su perfil desde su panel.

Qué se muestra: Búsqueda -> resultados -> perfil -> login profesional -> dashboard/servicios

Responsable: Lisandro (opera) / Mirian (narra)

4:30–5:30: Caso alternativo / validación

Qué se dice: El admin aprueba un perfil en 'en_revision' y ese perfil se vuelve público; un profesional inactivo no aparece en búsquedas.

Qué se muestra: admin -> aprobar en_revision; búsqueda sin el inactivo

Responsable: Lisandro (opera) / Juan (narra)

5:30–6:20: Decisión técnica / uso de IA

Qué se dice: PDO con prepared statements en toda la capa de datos, CSRF en los formularios y bcrypt para contraseñas. La IA propuso patrones; el equipo decidió y verificó.

Qué se muestra: funciones.php con sanitizar() y csrf_token()

Responsable: Juan

6:20–7:00: Estado actual y pendientes

Qué se dice: El MVP original está completo, incluyendo el formulario de solicitud de presupuesto. Reseñas y deploy son Fase 2.

Qué se muestra: Tabla de estado de funcionalidades (sección D)

Responsable: Mirian

P. Ensayo y verificación previa

☑ La versión de demo está respaldada (commit 0f3928d en main).

☑ Servidor, base de datos y servicios inician correctamente (XAMPP).

☑ El flujo principal fue probado de punta a punta (PF-1 a PF-12).

☑ Los datos de prueba ficticios están preparados (profesionales activos + 1 en revisión).

☑ No se muestran datos personales o secretos: las capturas usan perfiles de prueba (datos ficticios).

☑ Los integrantes conocen su rol.

☑ Guion completo: duración objetivo 5–7 minutos (bloques de duración variable).

☑ Existe un plan de contingencia.

☑ Las evidencias coinciden con la versión de demo.

☑ Los pendientes no se presentan como funcionalidades terminadas.

Q. Evidencias a incorporar

E1 – Qué muestra: Búsqueda por texto con resultados filtrados para “electricista”.

Qué criterio/caso respalda: PF-1: búsqueda funcional por oficio.

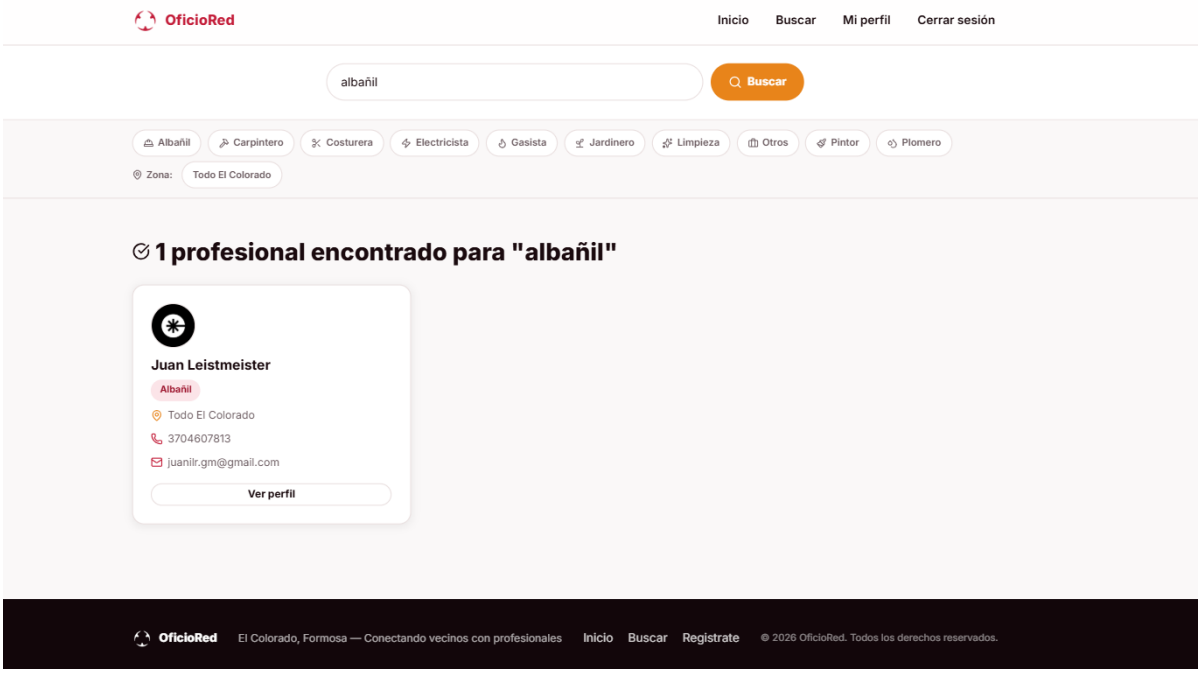


Figura E1. Búsqueda por texto con resultados filtrados para "electricista".

E2 – Qué muestra: Filtro por categoría desde el nav (chip "Albañil" activo) con resultados filtrados.

Qué criterio/caso respalda: PF-2: filtro por categoría.

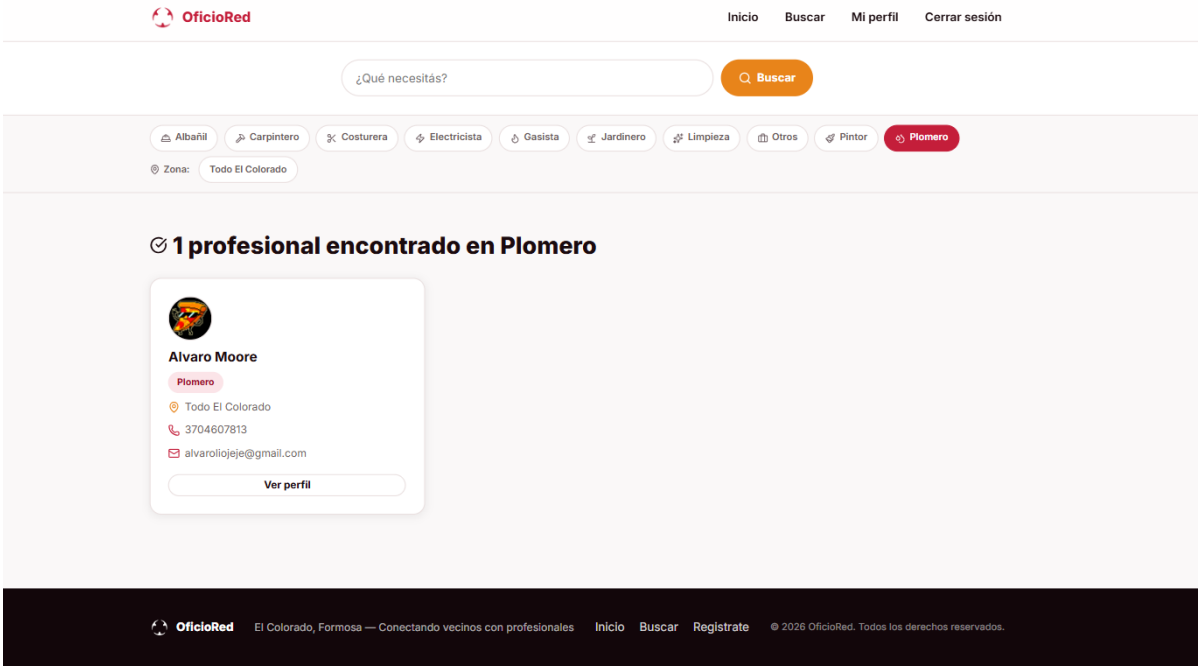


Figura E2. Filtro por categoría desde el nav (chip "Albañil" activo) con resultados filtrados.

E3 – Qué muestra: Perfil público de un profesional con oficio, zona, contacto y botón de WhatsApp.

Qué criterio/caso respalda: PF-5: perfil público completo.

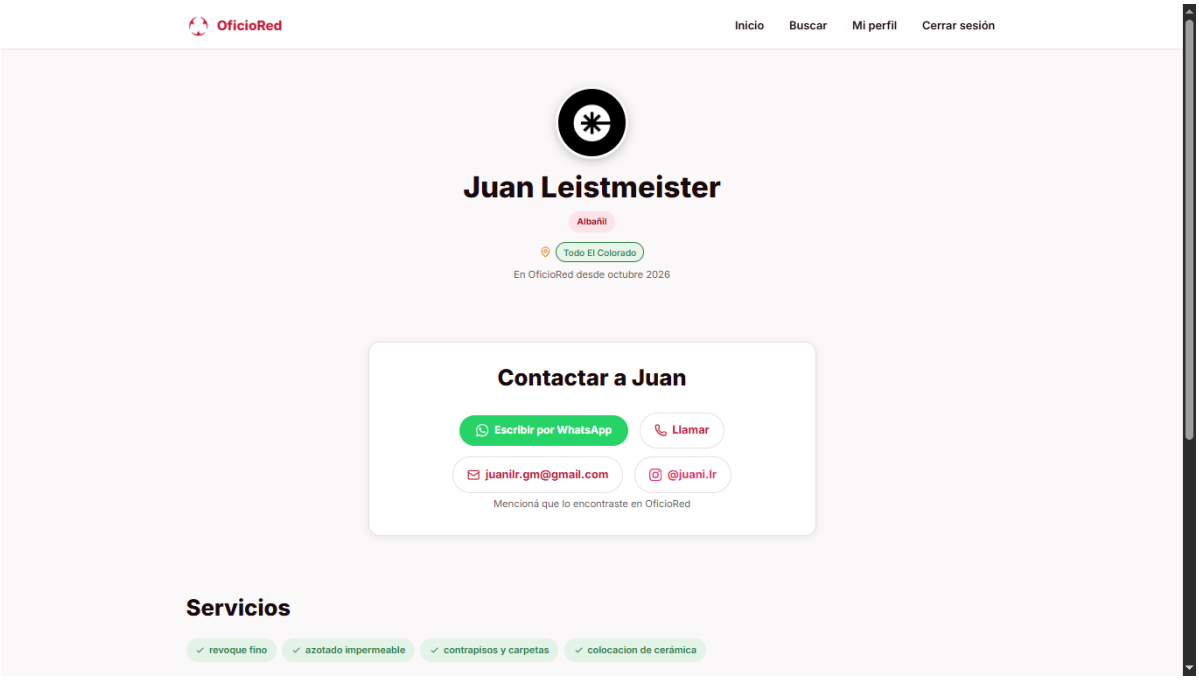


Figura E3. Perfil público de un profesional con oficio, zona, contacto y botón de WhatsApp.

E4 – Qué muestra: Dashboard del profesional autenticado con resumen y completitud del perfil.

Qué criterio/caso respalda: PF-4: login y dashboard funcional.

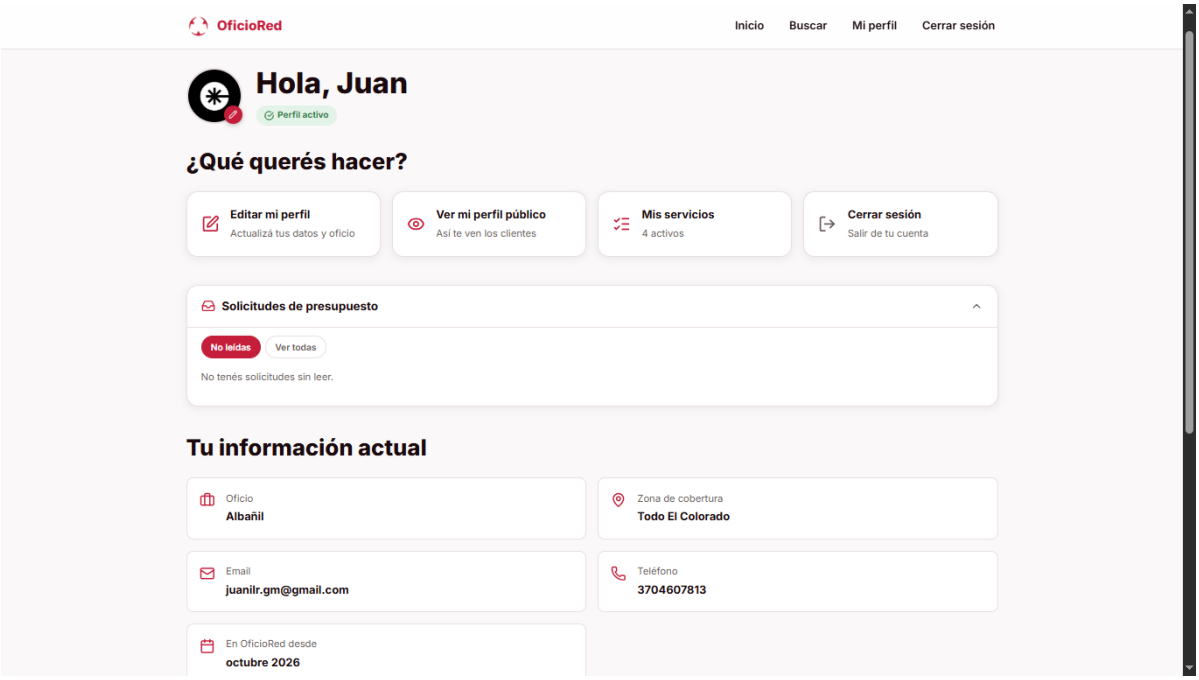


Figura E4. Dashboard del profesional autenticado con resumen y completitud del perfil.

E5 – Qué muestra: Panel de administración con profesional en estado en_revision y acciones de moderación (Aprobar/Rechazar/Eliminar).

Qué criterio/caso respalda: PF-6: flujo de moderación.

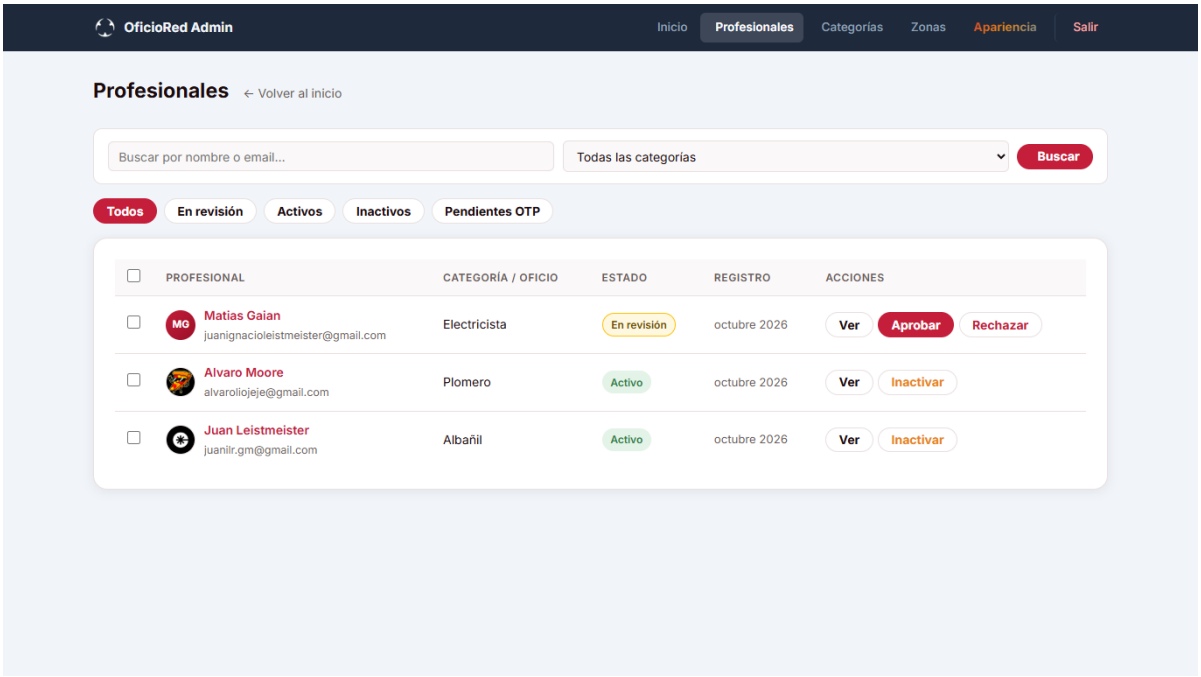


Figura E5. Panel de administración de profesionales con acciones de moderación (Ver/Activar/Eliminar).

E6 – Qué muestra: Panel de apariencia del sitio: configuración del hero (título, subtítulo, tipo de fondo, imagen).

Qué criterio/caso respalda: PF-11: apariencia configurable desde el admin.

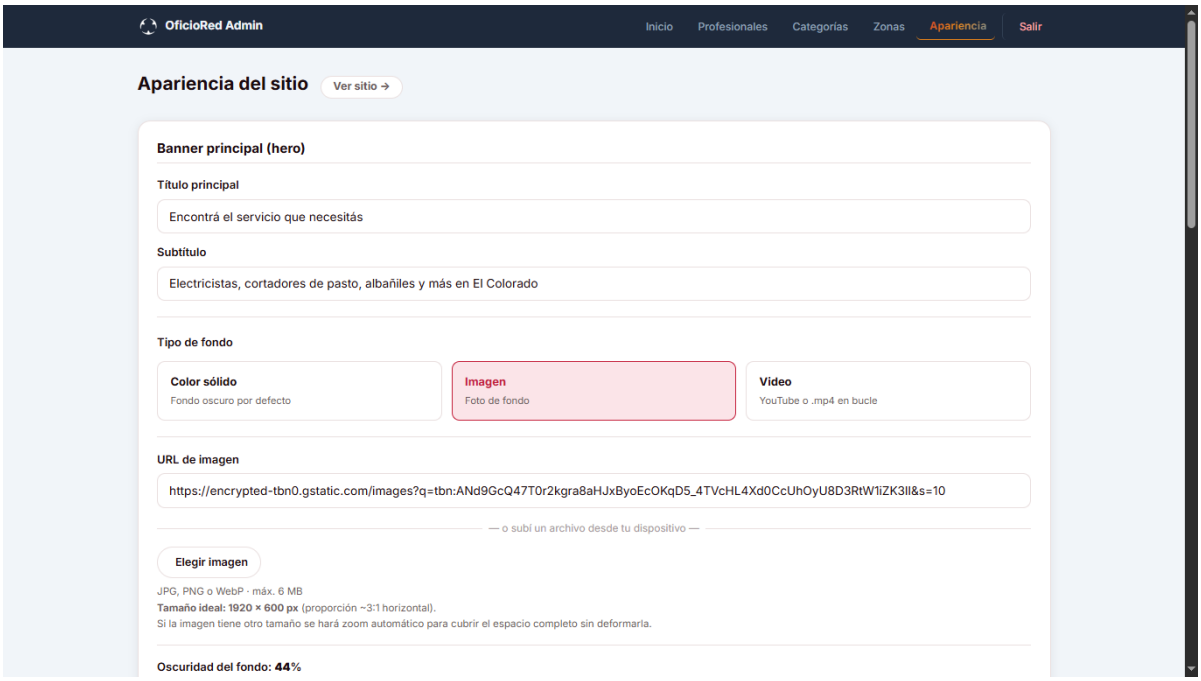


Figura E6. Panel de apariencia del sitio: configuración del hero (título, subtítulo, tipo de fondo, imagen).

E7 – Qué muestra: Formulario de solicitud de presupuesto con campos nombre, teléfono/WhatsApp, email, modalidad, zona y descripción del trabajo requerido.

Qué criterio/caso respalda: PF-11: apariencia configurable desde el admin.

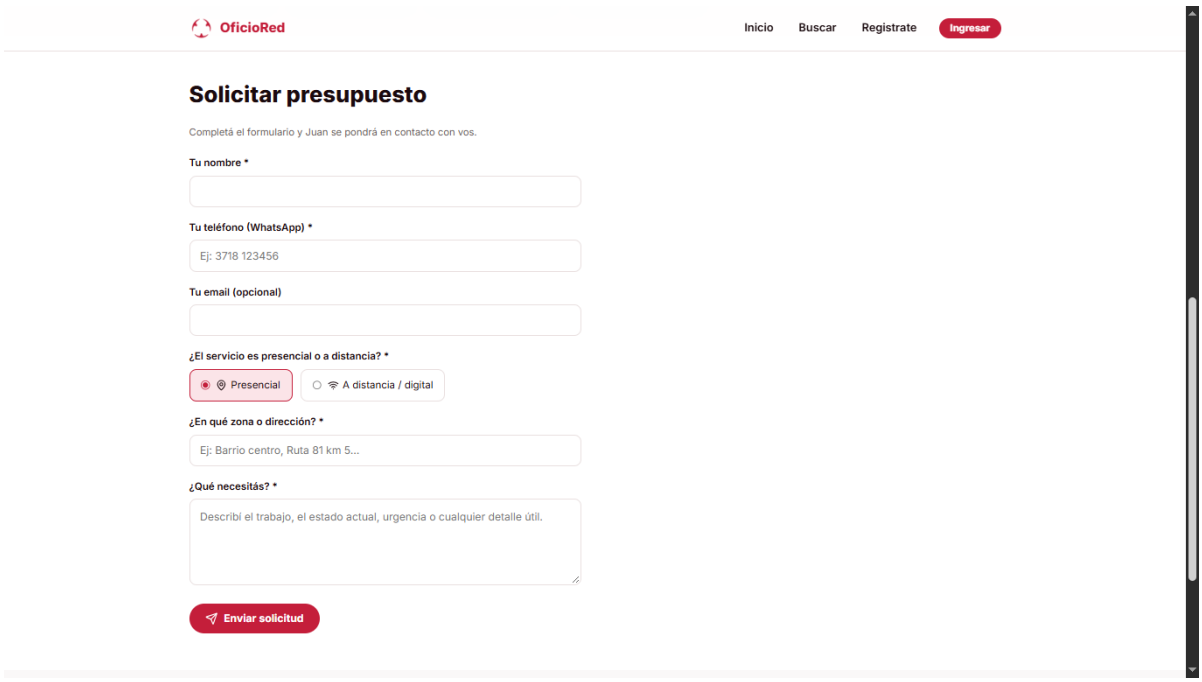


Figura E7. Formulario de solicitud de presupuesto: campos de contacto y descripción del trabajo.

R. Autoevaluación final del grupo

¿La documentación coincide con el sistema real?: Sí. Todo lo documentado fue verificado en entorno local sobre los 3 flujos.

¿Otra persona podría ejecutar el sistema con estas instrucciones?: Sí. La sección I y el README detallan pasos reproducibles con XAMPP.

¿La demo muestra un flujo funcional y no solo pantallas?: Sí. El guion (O.1) muestra los 3 flujos en ejecución real.

¿Las incidencias conocidas están declaradas?: Sí. La sección L lista incidencias y pendientes con estado y próxima acción.

¿El uso de IA está suficientemente documentado?: Sí. La sección M documenta 8 usos con la decisión tomada y su verificación.

¿Diferenciamos el MVP original del prototipo actual?: Sí. La sección B.1 separa MVP, prototipo y brecha (formulario de presupuesto).

Estado final para la demostración:

Listo.

Justificación: Todas las funcionalidades del MVP original implementadas, incluyendo formulario de solicitud de presupuesto (INC-01 resuelto). Evidencias E1-E7 incorporadas. INC-07 resuelto.

S. Checklist antes de entregar el TP12

☑ Usamos una versión real y estable del proyecto.

☑ No comenzamos una nueva feature para completar el TP.

☑ Diferenciamos MVP original y prototipo implementado.

☑ Actualizamos el README.

☑ Documentamos instalación y ejecución.

☑ Representamos arquitectura/flujo.

☑ Documentamos modelo de datos o persistencia.

☑ Registramos estado de funcionalidades.

☑ Incluimos pruebas y evidencias.

☑ Registramos incidencias y pendientes.

☑ Documentamos decisiones técnicas y uso de IA.

☑ Registramos rama/commit o respaldo.

☑ Preparamos guion de demo.

☑ Definimos roles.

☑ Preparamos datos ficticios.

☑ Ensayamos el guion con cronómetro.

☑ Definimos plan de contingencia.

☑ Verificamos que no haya credenciales o datos sensibles.

T. Entrega

Se entrega esta carpeta técnica en PDF (A4, con índice y numeración) junto con la referencia al repositorio y la versión estable (main, commit 0f3928d) y las evidencias E1–E7. Si durante la preparación de la demo se modifica código, se actualizarán también el commit de referencia, las pruebas y esta documentación.

CRITERIO DE CORRECCIÓN

Se evalúa la coherencia entre documentación, versión de código, evidencias, estado real del proyecto y guion de demostración.